Python Data Analysis Bootcamp

Hands-On Project with NumPy & Pandas

Complete Project Guide — Data Analysis & Reporting Edition

Level: Beginner to Intermediate

No visualization required — pure analysis and reporting

Project Overview

This project guides you through building a complete retail sales analytics and reporting pipeline using Python, NumPy, and Pandas. You will work with synthetic but realistic sales data to generate meaningful business insights, produce formatted summary reports, and export findings to Excel and text files. No visualization libraries are used — the focus is entirely on data analysis, aggregation, and professional text-based reporting.

Project Theme: Retail Sales Analytics — ShopSmart Inc.
You are a junior data analyst at ShopSmart Inc. Your manager needs a full written analysis of 12 months of sales transactions, delivered as formatted reports and Excel exports. Every output should be readable without charts.

Learning Objectives

- Master NumPy array operations, broadcasting, and statistical functions
- Build proficiency with Pandas DataFrames: filtering, groupby, merge, pivot, resample
- Apply data cleaning techniques including null handling and outlier treatment
- Perform exploratory data analysis (EDA) using only text-based outputs
- Generate formatted console reports, .txt summary files, and .xlsx exports
- Write well-structured, documented, reproducible Python code

Technologies Required

Library	Version	Purpose
Python	3.9+	Core programming language
NumPy	1.24+	Numerical computing and array operations
Pandas	2.0+	Data manipulation and analysis
openpyxl	3.1+	Write formatted .xlsx Excel reports
tabulate	0.9+	Print clean ASCII tables to console
Jupyter Notebook	6.5+	Interactive development environment

Installation

```
pip install numpy pandas openpyxl tabulate jupyter
jupyter notebook
```

Project Structure

```
shopsmart_analytics/
    data/
        sales_data.csv          # Raw transactions (generated Day 1)
        products.csv            # Product catalog
        customers.csv           # Customer profiles
        master_data.csv         # Cleaned merged dataset (Day 2)
    notebooks/
        day1_data_generation.ipynb
        day2_data_cleaning.ipynb
        day3_numpy_analysis.ipynb
        day4_pandas_analysis.ipynb
        day5_reporting.ipynb
    outputs/
        reports/
            executive_summary.txt
            category_report.txt
            customer_report.txt
        shopsmart_annual_report.xlsx
```

Project 1: Data Generation & Environment Setup

Today you will configure your Python environment and use NumPy's random number generator to produce three realistic CSV files. These files simulate a real-world retail database and will power every analysis through Project 5.

Goals for Project 1

- Set up the project folder structure
- Understand NumPy random number generation and seeding
- Generate products.csv, customers.csv, and sales_data.csv
- Perform an initial data inspection using Pandas

Concept: NumPy Random Module

NumPy's random module provides functions to generate arrays of numbers following statistical distributions. Setting a seed with np.random.seed() ensures that running the same script always produces the same data.

Key NumPy Functions Used Today

Function	Description	Example
np.random.seed(n)	Set seed for reproducibility	np.random.seed(42)
np.random.choice(arr, n)	Sample n items from array	np.random.choice(products, 1000)
np.random.normal(mu, sigma, n)	Normal distribution	np.random.normal(50, 15, 1000)
np.random.randint(low, high, n)	Random integers in range	np.random.randint(1, 100, 1000)
np.random.uniform(low, high, n)	Uniform float distribution	np.random.uniform(9.99, 499.99, n)
np.random.choice(arr, n, p=weights)	Weighted random choice	np.random.choice(tiers, 500, p=[0.4,0.3,0.2,0.1])

Step-by-Step Code: Data Generation

Step 1 — Imports and seed

```
import numpy as np
import pandas as pd
from datetime import datetime, timedelta
```

```
import os

np.random.seed(42)
N_TRANSACTIONS = 5000
os.makedirs('data', exist_ok=True)
os.makedirs('outputs/reports', exist_ok=True)
```

Step 2 — Generate product catalog

```
product_names = [
    'Laptop', 'Wireless Mouse', 'USB-C Hub', 'Monitor',
    'Keyboard', 'Webcam', 'Headphones', 'Phone Stand',
    'Desk Lamp', 'Notebook'
]
categories = [
    'Electronics', 'Electronics', 'Electronics', 'Electronics',
    'Electronics', 'Electronics', 'Audio', 'Accessories',
    'Accessories', 'Stationery'
]
base_prices = np.array([999.99, 29.99, 49.99, 349.99, 79.99,
                        89.99, 149.99, 19.99, 39.99, 9.99])

products_df = pd.DataFrame({
    'product_id': [f'P{i:03d}' for i in range(1, 11)],
    'product_name': product_names,
    'category': categories,
    'base_price': base_prices
})
products_df.to_csv('data/products.csv', index=False)
print('Products saved:', products_df.shape)
```

Step 3 — Generate customer profiles

```
N_CUSTOMERS = 500
cities = ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix']
tiers = ['Bronze', 'Silver', 'Gold', 'Platinum']

customers_df = pd.DataFrame({
    'customer_id': [f'C{i:04d}' for i in range(1, N_CUSTOMERS + 1)],
    'city': np.random.choice(cities, N_CUSTOMERS),
    'tier': np.random.choice(tiers, N_CUSTOMERS, p=[0.4, 0.3, 0.2,
0.1]),
    'age': np.random.randint(18, 75, N_CUSTOMERS),
    'join_year': np.random.randint(2018, 2024, N_CUSTOMERS)
})
customers_df.to_csv('data/customers.csv', index=False)
print('Customers saved:', customers_df.shape)
```

Step 4 — Generate sales transactions

```
start_date = datetime(2023, 1, 1)
date_offsets = np.random.randint(0, 365, N_TRANSACTIONS)
dates = [start_date + timedelta(days=int(d)) for d in date_offsets]

product_ids = np.random.choice(products_df['product_id'],
N_TRANSACTIONS)
customer_ids = np.random.choice(customers_df['customer_id'],
N_TRANSACTIONS)
```

```
quantities = np.random.choice(
    [1, 2, 3, 4, 5], N_TRANSACTIONS, p=[0.5, 0.25, 0.12, 0.08, 0.05]
)
discount_pct = np.random.choice(
    [0, 5, 10, 15, 20], N_TRANSACTIONS, p=[0.5, 0.2, 0.15, 0.1, 0.05]
)

sales_df = pd.DataFrame({
    'transaction_id': [f'T{i:05d}' for i in range(1, N_TRANSACTIONS +
1)],
    'date': dates,
    'product_id': product_ids,
    'customer_id': customer_ids,
    'quantity': quantities,
    'discount_pct': discount_pct
})

# Inject ~2% nulls for Project 2 cleaning practice
null_idx = np.random.choice(sales_df.index, size=100, replace=False)
sales_df.loc[null_idx[:50], 'quantity'] = np.nan
sales_df.loc[null_idx[50:], 'discount_pct'] = np.nan

sales_df.to_csv('data/sales_data.csv', index=False)
print('Sales saved:', sales_df.shape)
```

Step 5 — Initial inspection

```
print('\n--- SALES DATA PREVIEW ---')
print(sales_df.head(10).to_string())
print('\n--- COLUMN TYPES ---')
print(sales_df.dtypes)
print('\n--- MISSING VALUES ---')
print(sales_df.isnull().sum())
print('\n--- DESCRIPTIVE STATS ---')
print(sales_df.describe())
```

Expected Output Files

File	Rows	Columns	Description
products.csv	10	4	Product catalog with prices
customers.csv	500	5	Customer profiles with city and tier
sales_data.csv	5,000	6	Raw transactions with ~2% missing values

Project 1 Checkpoint Questions

- What is the purpose of np.random.seed(42)? What happens if you remove it?
- Run sales_df.describe() — what is the mean quantity and mean discount?
- How would you generate 10,000 transactions instead of 5,000?

- Which product has the highest base price? Use products_df.sort_values().

Project 2: Data Cleaning & Preprocessing

Real-world data is messy. Now you will apply essential data cleaning techniques using Pandas handling missing values, correcting data types, detecting outliers with NumPy, and merging all three datasets into one clean master DataFrame ready for analysis.

Goals for Project 2

- Inspect and understand data quality issues
- Handle missing values using appropriate fill strategies
- Parse dates and engineer time-based features
- Detect and treat outliers using the NumPy IQR method
- Merge products, customers, and sales into a master DataFrame
- Validate data integrity using assertions

The Data Cleaning Workflow

Step	Task	Key Tool
1. Inspect	Check shape, dtypes, null counts	df.info(), df.isnull().sum()
2. Fix Types	Parse dates, cast numerics	pd.to_datetime(), .astype()
3. Engineer Features	Extract month, quarter, day of week	dt.month, dt.quarter
4. Handle Nulls	Fill or drop missing values	.fillna(), .dropna()
5. Detect Outliers	IQR-based outlier flagging	np.percentile(), boolean masks
6. Merge Datasets	Join all three tables	pd.merge(how='left')
7. Compute Revenue	quantity * price * (1 - discount)	Vectorized arithmetic
8. Validate	Assert no nulls, no negatives, no dupes	assert, .duplicated()

Step-by-Step Code: Data Cleaning

Step 1 — Load and inspect

```
import numpy as np
import pandas as pd

sales = pd.read_csv('data/sales_data.csv')
products = pd.read_csv('data/products.csv')
customers = pd.read_csv('data/customers.csv')

print('Shape:', sales.shape)
print(sales.dtypes)
print('Nulls:\n', sales.isnull().sum())
```

Step 2 — Fix date type and engineer features

```
sales['date'] = pd.to_datetime(sales['date'])

sales['month']        = sales['date'].dt.month
sales['month_name']   = sales['date'].dt.strftime('%b')
sales['quarter']      = sales['date'].dt.quarter
sales['day_of_week']  = sales['date'].dt.day_name()
sales['week_number']  = sales['date'].dt.isocalendar().week.astype(int)
```

Step 3 — Handle missing values

```
# Fill quantity with median (robust to skew)
median_qty = sales['quantity'].median()
sales['quantity'] = sales['quantity'].fillna(median_qty).astype(int)

# Fill discount_pct with 0 (assume no discount if unknown)
sales['discount_pct'] = sales['discount_pct'].fillna(0)

print('Nulls after fill:', sales.isnull().sum().sum())
```

Step 4 — Merge all three datasets

```
# Left join: keep all sales rows
df = sales.merge(products, on='product_id', how='left')
df = df.merge(customers, on='customer_id', how='left')

print('Master shape:', df.shape)
print('Columns:', df.columns.tolist())
```

Step 5 — Compute revenue

```
# Revenue = quantity x base_price x (1 - discount/100)
df['revenue'] = (
    df['quantity'] * df['base_price'] *
    (1 - df['discount_pct'] / 100)
).round(2)
```

Step 6 — Detect outliers with NumPy IQR

```
revenue_arr = df['revenue'].values
Q1 = np.percentile(revenue_arr, 25)
Q3 = np.percentile(revenue_arr, 75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

```
outlier_mask = (df['revenue'] < lower_bound) | (df['revenue'] >
upper_bound)
print(f'Outliers: {outlier_mask.sum()} rows
({outlier_mask.mean()*100:.1f}%)')

# Cap outliers instead of dropping
df['revenue_capped'] = df['revenue'].clip(lower_bound, upper_bound)
```

Step 7 — Validate and save

```
assert df.isnull().sum().sum() == 0,        'Nulls remain!'
assert (df['revenue'] >= 0).all(),           'Negative revenue!'
assert not df.duplicated('transaction_id').any(), 'Duplicate IDs!'
print('All validation checks passed.')

df.to_csv('data/master_data.csv', index=False)
```

Project 2 Checkpoint Questions

- What is the difference between fillna() and dropna()? When is each appropriate?
- Why use median rather than mean to fill missing quantity values?
- What does a 'left' merge do? How many rows does the result have?
- After cleaning, how many transactions have a discount greater than 0?

Project 3: NumPy Deep Dive — Statistical Analysis

Today is devoted to NumPy. You will perform a thorough statistical analysis of revenue and quantity data using pure NumPy operations — without relying on Pandas aggregation methods. This teaches you what Pandas does under the hood and deepens your understanding of vectorized computation.

Goals for Project 3

- Apply NumPy descriptive statistics to real sales data
- Use NumPy boolean indexing to filter and segment data
- Implement broadcasting for multi-segment calculations
- Build a manual rolling average using np.convolve
- Compute correlations between numeric variables
- Print a clean statistical summary report to the console

NumPy Statistical Functions Reference

Function	Description	Use Case Today
np.mean(arr)	Arithmetic mean	Average revenue across all transactions
np.median(arr)	Middle value	Robust central tendency for skewed revenue
np.std(arr)	Standard deviation	Revenue variability / spread
np.var(arr)	Variance	Quantify dispersion
np.percentile(arr, q)	Nth percentile	IQR bounds, top/bottom deciles
np.corrcoef(x, y)	Pearson correlation matrix	Price vs quantity, discount vs revenue
np.cumsum(arr)	Cumulative sum	Running revenue total by month
np.convolve(arr, w, mode)	Convolution	3-month rolling average
np.argsort(arr)	Returns sorted indices	Rank products by revenue
np.where(cond, a, b)	Conditional element selection	Flag high-value transactions
np.unique(arr, return_counts=True)	Unique values and counts	Frequency of each product sold

np.ptp(arr)	Peak-to-peak (max - min)	Revenue range

Step-by-Step Code: NumPy Analysis

Setup — Load data as NumPy arrays

```
import numpy as np
import pandas as pd

df = pd.read_csv('data/master_data.csv', parse_dates=['date'])

# Extract NumPy arrays
revenue   = df['revenue'].values
quantity  = df['quantity'].values
prices    = df['base_price'].values
discounts = df['discount_pct'].values
months    = df['month'].values
```

Section A — Full Descriptive Statistics

```
def describe_array(arr, label):
    print(f'\n=== {label} ===')
    print(f'  Count   : {len(arr):>10,}')
    print(f'  Mean    : {np.mean(arr):>10.2f}')
    print(f'  Median  : {np.median(arr):>10.2f}')
    print(f'  Std Dev : {np.std(arr):>10.2f}')
    print(f'  Min     : {np.min(arr):>10.2f}')
    print(f'  Max     : {np.max(arr):>10.2f}')
    print(f'  Range   : {np.ptp(arr):>10.2f}')
    print(f'  P25     : {np.percentile(arr, 25):>10.2f}')
    print(f'  P75     : {np.percentile(arr, 75):>10.2f}')
    print(f'  P90     : {np.percentile(arr, 90):>10.2f}')

describe_array(revenue, 'REVENUE ($)')
describe_array(quantity, 'QUANTITY (units)')
describe_array(discounts, 'DISCOUNT (%)')
```

Section B — Monthly Revenue Aggregation

```
monthly_revenue = np.array([
    revenue[months == m].sum() for m in range(1, 13)
])

month_names = ['Jan','Feb','Mar','Apr','May','Jun',
               'Jul','Aug','Sep','Oct','Nov','Dec']

print('\n=== MONTHLY REVENUE ===')
for i, (name, rev) in enumerate(zip(month_names, monthly_revenue), 1):
    bar = '#' * int(rev / monthly_revenue.max() * 30)
    print(f'  {name}: ${rev:>10,.2f}  {bar}')

best_month_idx  = np.argmax(monthly_revenue)
worst_month_idx = np.argmin(monthly_revenue)
```

```
print(f'\nBest  month: {month_names[best_month_idx]}
(${monthly_revenue[best_month_idx]:,.2f})')
print(f'Worst month: {month_names[worst_month_idx]}
(${monthly_revenue[worst_month_idx]:,.2f})')
```

Section C — 3-Month Rolling Average

```
window  = 3
weights = np.ones(window) / window
rolling_avg = np.convolve(monthly_revenue, weights, mode='valid')

print('\n=== 3-MONTH ROLLING AVERAGE ===')
for i, avg in enumerate(rolling_avg):
    print(f'  {month_names[i+1]}: ${avg:>10,.2f}')
```

Section D — Broadcasting: Tier Multiplier

```
tier_map = {'Bronze': 1.0, 'Silver': 1.05, 'Gold': 1.10, 'Platinum':
1.15}
multiplier = df['tier'].map(tier_map).values

# NumPy broadcasting — no Python loops
adjusted_revenue = revenue * multiplier

print(f'\nOriginal total revenue  : ${revenue.sum():>12,.2f}')
print(f'Tier-adjusted revenue   : ${adjusted_revenue.sum():>12,.2f}')
print(f'Tier uplift             : ${adjusted_revenue.sum()-
revenue.sum():>12,.2f}')
```

Section E — Correlation Matrix

```
corr = np.corrcoef([prices, quantity, discounts])

labels = ['Price', 'Quantity', 'Discount']
print('\n=== PEARSON CORRELATION MATRIX ===')
header = f'  {'':12s}' + ''.join(f'{l:>12s}' for l in labels)
print(header)
for i, row in enumerate(corr):
    print(f'  {labels[i]:12s}' + ''.join(f'{v:>12.3f}' for v in row))
```

Section F — Top 10 Highest Revenue Transactions

```
top10_idx = np.argsort(revenue)[-10:][::-1]

print('\n=== TOP 10 REVENUE TRANSACTIONS ===')
for rank, idx in enumerate(top10_idx, 1):
    tid = df.iloc[idx]['transaction_id']
    rev = revenue[idx]
    prod = df.iloc[idx]['product_name']
    print(f'  {rank:>2}. {tid}  {prod:20s}  ${rev:>8,.2f}')
```

Section G — Flag high-value transactions with np.where

```
threshold = np.percentile(revenue, 90)
df['value_segment'] = np.where(revenue >= threshold, 'High-Value',
'Standard')

high_val_count = (df['value_segment'] == 'High-Value').sum()
```

```
high_val_rev   = revenue[revenue >= threshold].sum()
print(f'\nHigh-value transactions (top 10%): {high_val_count}')
print(f'Revenue from high-value transactions: ${high_val_rev:,.2f}')
```

Project 3 Checkpoint Questions

- What is the difference between np.mean() and np.median()? Which is more appropriate for skewed revenue data?
- What does np.convolve() do? Why use mode='valid' for rolling averages?
- Is there a meaningful correlation between discount percentage and quantity purchased?
- What percentage of total revenue comes from the top 10% of transactions?

Project 4: Pandas Deep Dive — Business Insights

Today you harness Pandas' most powerful analytical tools to answer five concrete business questions. You will use groupby with multiple aggregations, pivot tables, window functions, and advanced filtering to produce structured tabular outputs ready for the Project 5 report.

Goals for Project 4

- Use groupby with named aggregations to summarize KPIs
- Build pivot tables for multidimensional cross-tabulation
- Apply window functions: rolling, pct_change, rank, cumsum
- Use pd.crosstab for frequency analysis
- Save all analysis results to CSV files in outputs/

The 6 Business Questions

#	Business Question	Technique
1	Which category drives the most revenue and volume?	groupby + named agg
2	Who are the top 10 customers by lifetime value?	groupby + sort_values + head
3	How does revenue trend month-over-month?	resample + pct_change + cumsum
4	Does customer tier affect average order value?	pivot_table
5	What is the best product in each city?	groupby + idxmax
6	What is the impact of discount levels on revenue?	cut + groupby

Step-by-Step Code: Pandas Analysis

Setup

```
import pandas as pd
import numpy as np
import os

df = pd.read_csv('data/master_data.csv', parse_dates=['date'])
os.makedirs('outputs', exist_ok=True)
```

Q1 — Revenue and Volume by Category

```
category_summary = (
    df.groupby('category')
    .agg(
        total_revenue     = ('revenue', 'sum'),
        avg_order_value   = ('revenue', 'mean'),
        total_units       = ('quantity', 'sum'),
        num_transactions  = ('transaction_id', 'count'),
        unique_customers  = ('customer_id', 'nunique'),
        avg_discount      = ('discount_pct', 'mean')
    )
    .sort_values('total_revenue', ascending=False)
    .round(2)
)

# Add revenue share column
category_summary['revenue_share_pct'] = (
    category_summary['total_revenue'] /
    category_summary['total_revenue'].sum() * 100
).round(1)

print(category_summary.to_string())
category_summary.to_csv('outputs/category_summary.csv')
```

Q2 — Top 10 Customers by Lifetime Value

```
customer_ltv = (
    df.groupby(['customer_id', 'city', 'tier'])
    .agg(
        lifetime_value  = ('revenue', 'sum'),
        total_orders    = ('transaction_id', 'count'),
        avg_order_value = ('revenue', 'mean'),
        total_units     = ('quantity', 'sum'),
        first_purchase  = ('date', 'min'),
        last_purchase   = ('date', 'max'),
    )
    .reset_index()
    .sort_values('lifetime_value', ascending=False)
    .head(10)
    .round(2)
)

print(customer_ltv[['customer_id','city','tier',

'lifetime_value','total_orders']].to_string(index=False))
customer_ltv.to_csv('outputs/top10_customers.csv', index=False)
```

Q3 — Month-over-Month Revenue Trend

```
monthly = (
    df.set_index('date')
    .resample('ME')
    .agg(
        monthly_revenue  = ('revenue', 'sum'),
        num_transactions = ('transaction_id', 'count'),
        avg_order_value  = ('revenue', 'mean'),
        total_units      = ('quantity', 'sum')
    )
```

```
)

monthly['mom_change_pct'] = monthly['monthly_revenue'].pct_change() *
100
monthly['cum_revenue']    = monthly['monthly_revenue'].cumsum()
monthly['rolling_3m_avg'] =
monthly['monthly_revenue'].rolling(3).mean()
monthly = monthly.round(2)

print(monthly.to_string())
monthly.to_csv('outputs/monthly_trend.csv')
```

Q4 — Tier vs Average Order Value (Pivot Table)

```
tier_pivot = pd.pivot_table(
    df,
    values  = 'revenue',
    index   = 'tier',
    columns = 'quarter',
    aggfunc = {'revenue': ['mean', 'sum', 'count']},
    margins = True,
    margins_name = 'All Tiers'
).round(2)

print(tier_pivot.to_string())
tier_pivot.to_csv('outputs/tier_pivot.csv')
```

Q5 — Best Product per City

```
city_product_rev = (
    df.groupby(['city', 'product_name'])['revenue']
    .sum()
    .unstack(fill_value=0)
)

best_product_per_city = pd.DataFrame({
    'best_product'    : city_product_rev.idxmax(axis=1),
    'top_revenue'     : city_product_rev.max(axis=1).round(2),
    'num_products_sold': (city_product_rev > 0).sum(axis=1)
})

print(best_product_per_city.to_string())
best_product_per_city.to_csv('outputs/best_product_per_city.csv')
```

Q6 — Discount Level Impact on Revenue

```
# Bin discount levels
df['discount_band'] = pd.cut(
    df['discount_pct'],
    bins=[-1, 0, 5, 10, 15, 20],
    labels=['No Discount', '1-5%', '6-10%', '11-15%', '16-20%']
)

discount_impact = (
    df.groupby('discount_band', observed=True)
    .agg(
        num_transactions = ('transaction_id', 'count'),
```

```
        total_revenue    = ('revenue', 'sum'),
        avg_revenue      = ('revenue', 'mean'),
        avg_quantity     = ('quantity', 'mean'),
    )
    .round(2)
)

print(discount_impact.to_string())
discount_impact.to_csv('outputs/discount_impact.csv')
```

Bonus — Product Performance Matrix

```
product_matrix = (
    df.groupby(['product_name', 'category'])
    .agg(
        units_sold        = ('quantity', 'sum'),
        total_revenue     = ('revenue', 'sum'),
        avg_discount      = ('discount_pct', 'mean'),
        unique_customers  = ('customer_id', 'nunique'),
        num_transactions  = ('transaction_id', 'count'),
    )
    .sort_values('total_revenue', ascending=False)
    .round(2)
)
product_matrix['revenue_per_unit'] = (
    product_matrix['total_revenue'] / product_matrix['units_sold']
).round(2)

print(product_matrix.to_string())
product_matrix.to_csv('outputs/product_matrix.csv')
```

Project 4 Checkpoint Questions

- What does pct_change() compute, and in which months was growth negative?
- Which product category has the highest average order value (not total revenue)?
- Does the Platinum tier consistently outperform Bronze tier across all quarters?
- At which discount level does average quantity purchased peak?

Project 5: Report Generation & Excel Export

Today you bring the entire project together. You will write a multi-section Python reporting engine that reads all the analysis outputs from Project 4 and produces three formatted text reports, one comprehensive Excel workbook with multiple styled sheets, and a final executive summary. Pure data storytelling through numbers and structured text.

Goals for Project 5

- Build a reusable report_writer() function that formats tables with tabulate
- Generate three standalone .txt report files (Executive Summary, Category Report, Customer Report)
- Export a multi-sheet .xlsx workbook using pandas ExcelWriter + openpyxl
- Apply column width auto-sizing and basic formatting to Excel sheets
- Print a final console summary with all key KPIs

Report Output Plan

Output File	Contents	Format
executive_summary.txt	KPIs, top metrics, month-over-month highlights	Plain text, formatted headers
category_report.txt	Full category breakdown with revenue share	ASCII table via tabulate
customer_report.txt	Top 10 customers with LTV analysis	ASCII table via tabulate
shopsmart_annual_report.xlsx	6 sheets: Summary, Categories, Customers, Monthly, Products, Discounts	Excel with auto-sized columns

Step-by-Step Code: Report Engine

Setup and helpers

```
import pandas as pd
import numpy as np
from tabulate import tabulate
from openpyxl import load_workbook
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from openpyxl.utils import get_column_letter
import os
import datetime

# Load master data and all pre-computed outputs
```

```
df              = pd.read_csv('data/master_data.csv',
parse_dates=['date'])
category_sum    = pd.read_csv('outputs/category_summary.csv',
index_col=0)
top_customers   = pd.read_csv('outputs/top10_customers.csv')
monthly         = pd.read_csv('outputs/monthly_trend.csv', index_col=0)
product_matrix  = pd.read_csv('outputs/product_matrix.csv')
discount_impact = pd.read_csv('outputs/discount_impact.csv')
best_per_city   = pd.read_csv('outputs/best_product_per_city.csv',
index_col=0)

os.makedirs('outputs/reports', exist_ok=True)
```

Helper: ASCII section divider

```
def section(title, width=60):
    line = '=' * width
    return f'\n{line}\n  {title}\n{line}'
```

Report 1 — Executive Summary

```
total_revenue     = df['revenue'].sum()
total_transactions = len(df)
avg_order_value   = df['revenue'].mean()
total_units       = df['quantity'].sum()
unique_customers  = df['customer_id'].nunique()
best_category     = category_sum['total_revenue'].idxmax()
best_month_idx    = monthly['monthly_revenue'].idxmax()
best_customer_id  = top_customers.iloc[0]['customer_id']
best_customer_ltv = top_customers.iloc[0]['lifetime_value']

# Month-over-month changes
mom = monthly['mom_change_pct'].dropna()
positive_months = (mom > 0).sum()
negative_months = (mom < 0).sum()
best_growth_month  = mom.idxmax()
worst_growth_month = mom.idxmin()

report_date = datetime.date.today().strftime('%B %d, %Y')

exec_summary = f'''
SHOPSMART INC. — ANNUAL SALES REPORT 2023
Report Generated: {report_date}
Prepared by: Data Analytics Team
{section('FINANCIAL HIGHLIGHTS')}
  Total Revenue         : ${total_revenue:>12,.2f}
  Total Transactions    : {total_transactions:>12,}
  Total Units Sold      : {total_units:>12,}
  Average Order Value   : ${avg_order_value:>12,.2f}
  Unique Customers      : {unique_customers:>12,}
{section('TOP PERFORMERS')}
  Best Category         : {best_category}
  Best Month            : {best_month_idx}
  Top Customer          : {best_customer_id} (LTV:
${best_customer_ltv:,.2f})
{section('MONTH-OVER-MONTH GROWTH')}
  Months with growth    : {positive_months} of 11
```

```
  Months with decline   : {negative_months} of 11
  Strongest growth month: {best_growth_month}
  Weakest growth month  : {worst_growth_month}
{section('REVENUE BY CATEGORY')}
'''

# Append category table
cat_display =
category_sum[['total_revenue','revenue_share_pct','num_transactions']].
copy()
cat_display.columns = ['Total Revenue ($)', 'Share (%)',
'Transactions']
exec_summary += tabulate(cat_display, headers='keys', tablefmt='grid',
                         floatfmt='.2f') + '\n'

# Save report
with open('outputs/reports/executive_summary.txt', 'w') as f:
    f.write(exec_summary)
print('Executive summary saved.')
```

Report 2 — Category Detailed Report

```
cat_report = section('SHOPSMART INC. — CATEGORY PERFORMANCE REPORT
2023')

cat_report += '\n\n1. FULL CATEGORY BREAKDOWN\n'
cat_report += tabulate(
    category_sum.reset_index(),
    headers=['Category','Revenue','Avg Order','Units','Transactions',
             'Customers','Avg Discount','Share %'],
    tablefmt='grid', floatfmt='.2f'
) + '\n'

# Product-level breakdown
cat_report += '\n2. PRODUCT PERFORMANCE WITHIN CATEGORIES\n'
for cat, grp in product_matrix.groupby('category'):
    cat_report += f'\n  [{cat}]\n'
    sub = grp[['product_name','units_sold','total_revenue',
               'revenue_per_unit','unique_customers']].copy()
    cat_report += tabulate(sub, headers='keys', tablefmt='simple',
                           showindex=False, floatfmt='.2f') + '\n'

with open('outputs/reports/category_report.txt', 'w') as f:
    f.write(cat_report)
print('Category report saved.')
```

Report 3 — Customer Report

```
cust_report  = section('SHOPSMART INC. — CUSTOMER REPORT 2023')
cust_report += '\n\nTOP 10 CUSTOMERS BY LIFETIME VALUE\n'
cust_report += tabulate(
    top_customers[['customer_id','city','tier','lifetime_value',
                   'total_orders','avg_order_value']],
    headers=['ID','City','Tier','LTV ($)','Orders','Avg Order ($)'],
    tablefmt='grid', showindex=False, floatfmt='.2f'
) + '\n'
```

```
# Tier summary
tier_summary = df.groupby('tier').agg(
    customers   = ('customer_id', 'nunique'),
    total_rev   = ('revenue', 'sum'),
    avg_order   = ('revenue', 'mean'),
    total_orders= ('transaction_id', 'count')
).round(2)
cust_report += '\nCUSTOMER TIER SUMMARY\n'
cust_report += tabulate(tier_summary, headers='keys',
                        tablefmt='grid', floatfmt='.2f') + '\n'

with open('outputs/reports/customer_report.txt', 'w') as f:
    f.write(cust_report)
print('Customer report saved.')
```

Excel Export — Multi-Sheet Workbook

```
excel_path = 'outputs/shopsmart_annual_report.xlsx'

with pd.ExcelWriter(excel_path, engine='openpyxl') as writer:

    # Sheet 1: Executive KPIs
    kpi_data = pd.DataFrame({
        'Metric': ['Total Revenue', 'Total Transactions', 'Total
Units',
                   'Avg Order Value', 'Unique Customers', 'Best
Category',
                   'Best Month', 'Top Customer'],
        'Value': [f'${total_revenue:,.2f}', f'{total_transactions:,}',
                  f'{total_units:,}', f'${avg_order_value:.2f}',
                  f'{unique_customers:,}', best_category,
                  str(best_month_idx), best_customer_id]
    })
    kpi_data.to_excel(writer, sheet_name='Executive Summary',
index=False)

    # Sheet 2: Category Summary
    category_sum.reset_index().to_excel(
        writer, sheet_name='Category Analysis', index=False
    )

    # Sheet 3: Top Customers
    top_customers.to_excel(writer, sheet_name='Top Customers',
index=False)

    # Sheet 4: Monthly Trend
    monthly.reset_index().to_excel(writer, sheet_name='Monthly Trend',
index=False)

    # Sheet 5: Product Matrix
    product_matrix.reset_index().to_excel(
        writer, sheet_name='Product Performance', index=False
    )

    # Sheet 6: Discount Analysis
    discount_impact.reset_index().to_excel(
```

```
            writer, sheet_name='Discount Analysis', index=False
        )
```

Apply Excel formatting with openpyxl

```
wb = load_workbook(excel_path)

header_fill = PatternFill('solid', fgColor='2B5797')
header_font = Font(bold=True, color='FFFFFF', size=11)
alt_fill    = PatternFill('solid', fgColor='EEF3FB')
thin        = Side(border_style='thin', color='CCCCCC')
cell_border = Border(left=thin, right=thin, top=thin, bottom=thin)

for sheet_name in wb.sheetnames:
    ws = wb[sheet_name]

    # Style header row
    for cell in ws[1]:
        cell.fill      = header_fill
        cell.font      = header_font
        cell.alignment = Alignment(horizontal='center',
vertical='center')
        cell.border    = cell_border

    # Alternate row shading and borders for data rows
    for row_idx, row in enumerate(ws.iter_rows(min_row=2), start=2):
        fill = alt_fill if row_idx % 2 == 0 else PatternFill()
        for cell in row:
            cell.fill   = fill
            cell.border = cell_border

    # Auto-size columns
    for col in ws.columns:
        max_len = max(
            len(str(c.value)) if c.value else 0 for c in col
        )
        ws.column_dimensions[get_column_letter(col[0].column)].width =
min(max_len + 4, 40)

wb.save(excel_path)
print(f'Excel workbook saved: {excel_path}')
```

Final console summary

```
print(section('PROJECT COMPLETE — FINAL SUMMARY', width=70))
print(f'  Text Reports  : outputs/reports/ (3 files)')
print(f'  Excel Report  : outputs/shopsmart_annual_report.xlsx')
print(f'  Sheets        : {len(wb.sheetnames)} sheets with formatted
data')
print(f'  Total Revenue : ${total_revenue:,.2f}')
print(f'  Transactions  : {total_transactions:,}')
print(f'  Avg Order     : ${avg_order_value:.2f}')
print('=' * 70)
```

Project 5 Checkpoint Questions

- What does the tabulate library do, and what tablefmt options are available (grid, simple, pipe, latex)?
- What is the difference between pd.ExcelWriter and directly calling df.to_excel()?
- In the openpyxl formatting loop, why use get_column_letter() instead of a column index?
- Open the Excel file — do all 6 sheets display correctly with headers formatted in blue?

Project Summary & Extension Ideas

What You Accomplished in 5 Projects

Day	Topic	Key Output
Project 1	Data Generation	3 CSV files: 5,000 transactions, 500 customers, 10 products
Project 2	Data Cleaning	Cleaned master_data.csv with computed revenue column
Project 3	NumPy Statistics	Statistical summaries, rolling averages, correlation matrix
Project 4	Pandas Analysis	6 business question analyses, all saved to CSV
Project 5	Reporting & Export	3 text reports + 1 Excel workbook with 6 formatted sheets

Skills Gained

Skill Area	Topics Covered
NumPy Fundamentals	Arrays, broadcasting, statistical functions, argsort, convolve, where, corrcoef
Pandas Wrangling	read_csv, merge, fillna, astype, groupby, agg, pivot_table, resample, cut
Data Cleaning	Null detection and handling, outlier IQR method, type casting, validation assertions
Reporting	tabulate ASCII tables, formatted f-string reports, .txt file writing
Excel Export	pd.ExcelWriter, multi-sheet workbooks, openpyxl header styling, column auto-sizing
Software Practices	Random seeds for reproducibility, assertions, folder management, code organisation

Extension Challenges

Once you have completed all five Projects, push your skills further with these challenges:

- Challenge 1 (Beginner): Add a returns table and compute net revenue. Re-run all Day 4 analyses using net_revenue instead of revenue.
- Challenge 2 (Intermediate): Use scipy.stats.ttest_ind to test whether Gold-tier customers have a statistically significantly higher average order value than Bronze-tier customers. Print the p-value and conclusion.
- Challenge 3 (Intermediate): Use np.polyfit to fit a linear trend to monthly revenue and forecast the next 3 months. Print the forecast values in the executive summary report.
- Challenge 4 (Advanced): Refactor the entire project into a Python class ShopSmartAnalyzer with methods: load_data(), clean(), run_numpy_analysis(), run_pandas_analysis(), and generate_reports(). Each method should save its own outputs.
- Challenge 5 (Advanced): Add a cohort analysis — group customers by their join_year and calculate average lifetime value, average number of orders, and retention rate for each cohort. Export to an additional Excel sheet.

Congratulations — you have built a complete, professional-grade data analysis and reporting pipeline in Python. You now have a full portfolio project demonstrating NumPy, Pandas, data cleaning, business analytics, and automated report generation.